

Tools in LLM

Tools in LLM

Did you know? In Python, `**` in a function call means take this mapping (usually a dict) and expand it into keyword arguments.

What's happening in your call

Python sees a dict with keys `a` and `b`, and expands it as if you had written:

So its not passing the dict as a single argument. Its unpacking it into named parameters.

Why it works

Because your function is defined with parameter names `a` and `b`:

and the dict has matching keys:

So `add_something` gets `a`, gets `b`.

Rules (the important bits)

1. Keys must be strings (valid keyword names, effectively):
2. Keys must match parameter names, unless the function accepts arbitrary keywords:
3. You can mix normal args + `*` (but dont give the same value twice):
4. You can also merge multiple `**` expansions:

Related: `*` vs `**`

* unpacks a sequence into positional arguments:

* unpacks a mapping into keyword arguments:

If you want, I can show how this interacts with `kwargs` which is where `**` really becomes obvious.

Code

```
def add_something(a, b):
    print(f"Adding {a} and {b} together...")
    return a + b
add_something(*[7, 12])
```

Output

Adding 7 and 12 together...

Output

19

Code

```
import os, json
from dotenv import load_dotenv
import textwrap
def pretty_print(*args):
    text = " ".join(str(arg) for arg in args)
    try:
        print(textwrap.fill(text, width=80))
    except Exception as e:
        print(text) # fallback to normal print if text is not a string
    load_dotenv('/Users/shivam13juna/Documents/scaler/iitr_classes/llm_ref/openai_key.env') #
    reads .env file in the current directory
    api_key = os.getenv("OPENAI_API_KEY")
    if not api_key:
        raise ValueError(
            "OPENAI_API_KEY not found! "
            "Make sure you have a .env file with: OPENAI_API_KEY=sk-..."
        )
    pretty_print("API key loaded successfully.")
```

Output

```
API key loaded successfully.
```

Code

```
import truststore
truststore.inject_into_ssl()
#This is optional. I use VPN in my computer. Why I have to use this
```

Code

```
from openai import OpenAI
client = OpenAI(api_key=api_key)
pretty_print("OpenAI client ready.")
MODEL = "gpt-5-nano"
```

Output

```
OpenAI client ready.
```

Code

```
# The model has NO access to live data - what does it do?
response = client.responses.create(
    model=MODEL,
    input="What is the weather in bengaluru right now? Give me the exact temperature.",
    reasoning={"effort": "minimal"},
    text={"verbosity": "low"}
)
print(response.output_text)
```

Output

I can't access live weather data. Please check a weather app or website (e.g., [Weather.com](https://weather.com),

Google Weather) for the exact current temperature in Bengaluru. If you tell me your preferred

source, I can guide you how to find it there.

Code

```
# The model PREDICTS math - it doesn't COMPUTE it
response = client.responses.create(
    model=MODEL,
    input="What is 1247 * 83 + 19 / 3.7? Give me answer in one line (max 10 words), the exact
    number, upto 2 decimal places.",
    reasoning={"effort": "minimal"},
    text={"verbosity": "low"}
)
print("Model says:", response.output_text)
# What Python actually computes
import math
actual = 1247 * 83 + 19 / 3.7
print(f"Actual answer: {actual}")
```

Output

```
Model says: 103,601 + 5.135135... = 103,606.13514  
Actual answer: 103506.13513513513
```

Code

```
def get_weather(location):  
    # Simulate fetching weather data  
    return f"The current temperature in {location} is 28°C with clear skies."
```

Code

```

# Define a simple "add" tool - we'll explain the structure in detail next
add_tool = {
    "type": "function",
    "name": "add",
    "description": "Add two numbers together a and b, and return the sum.",
    "parameters": {
        "type": "object",
        "properties": {
            "a": {"type": "number", "description": "First number"},
            "b": {"type": "number", "description": "Second number"},
        },
        "required": ["a", "b"],
        "additionalProperties": False,
    },
    "strict": True,
}

# Ask the model to add - but give it the tool
response = client.responses.create(
    model=MODEL,
    instructions="Never compute math yourself.",
    input="What is '7' + '12'?",
    tools=[add_tool],
)

# Let's inspect what came back
print("Output items from the model:")
print("-" * 40)
for item in response.output:
    print(f"  Type: {item.type}")
    if item.type == "function_call":
        print(f"  Function name: {item.name}")
        print(f"  Arguments: {item.arguments}")
        print(f"  Call ID: {item.call_id}")
    pretty_print(" Response: " + response.output_text)

```


Output

Output items from the model:

Type: reasoning
Type: function_call
Function name: add
Arguments: {"a":7,"b":12}
Call ID: call_jGJghi8JSoSqre8JexpT14kk
Response:

Code

```

import json
# Define a simple "add" tool - we'll explain the structure in detail next
add_tool = {
    "type": "function",
    "name": "addy",
    "description": "Add two numbers together a and b, and return the sum.",
    "parameters": {
        "type": "object",
        "properties": {
            "a": {"type": "number", "description": "First number"},
            "b": {"type": "number", "description": "Second number"},
        },
        "required": ["a", "b"],
        "additionalProperties": False,
    },
    "strict": True,
}
# Step 0: Our actual Python function (the model never sees this!)
def add(a, b):
    return a + b
# Step 1: Call the API with tools defined
response = client.responses.create(
    model=MODEL,
    instructions="Always use the add tool for math. Never compute yourself.",
    input="What is 7 + 12?",
    tools=[add_tool], # from earlier
)
print("??? Step 2: Model returns a function_call (not text!) ???")
for item in response.output:
    print(f" {item.type}: ", end="")
    if item.type == "function_call":
        print(f"{item.name}({item.arguments})")
    else:
        print(getattr(item, 'content', ''))

```

Output

```
??? Step 2: Model returns a function_call (not text!) ???  
reasoning: None  
function_call: addy({"a":7, "b":12})
```

Code

```
# Step 3: We execute the function with the model's arguments  
function_call = [item for item in response.output if item.type == "function_call"][0]  
function_call
```

Output

```
ResponseFunctionToolCall(arguments='{"a":7, "b":12}', call_id='call_mk0q18aIU5x9NLZ4M8dChz8g',
```

name='addy', type='function_call', id='fc_07b1d62bbdbe538c0069e10e89b964819890175b03084ff2ae',

```
namespace=None, status='completed')
```

Code

```
json.loads(function_call.arguments)
```

Output

```
{'a': 7, 'b': 12}
```

Code

```
args = json.loads(function_call.arguments)
```

Code

```
mapping = {'addy': add}
```

Code

```
result = mapping(**args)
result
```

Output

```
19
```

Code

```
followup_input = []
followup_input.append({
    "type": "function_call_output",      # this is OUR result going back
    "call_id": function_call.call_id,    # links to the specific function_call
    "output": str(result),               # must be a string
})
# Step 5: Model writes the final answer using the real result
final = client.responses.create(
    model=MODEL,
    instructions="Always use the add tool for math. Never compute yourself.",
    previous_response_id=response.id,    # links to the previous response
    input=followup_input,
    tools=[add_tool],
)
print(f"??? Step 5: Final answer ???")
print(final.output_text)
```

Output

```
??? Step 5: Final answer ???  
7 + 12 = 19
```

Let's build calculator

Code

```

# Adding more tools doesn't change the model's behavior if the question doesn't require them
add_tool = {
    "type": "function",
    "name": "add",
    "description": "Add two numbers together and return the sum.",
    "parameters": {
        "type": "object",
        "properties": {
            "a": {"type": "number", "description": "First number"},
            "b": {"type": "number", "description": "Second number"},
        },
        "required": ["a", "b"],
        "additionalProperties": False,
    },
    "strict": True,
}
sub_tool = {
    "type": "function",
    "name": "subtract",
    "description": "Subtract two numbers and return the difference.",
    "parameters": {
        "type": "object",
        "properties": {
            "a": {"type": "number", "description": "First number"},
            "b": {"type": "number", "description": "Second number"},
        },
        "required": ["a", "b"],
        "additionalProperties": False,
    },
    "strict": True,
}
mul_tool = {
    "type": "function",
    "name": "multiply",
    "description": "Multiply two numbers together and return the product.",
    "parameters": {
        "type": "object",
        "properties": {
            "a": {"type": "number", "description": "First number"},
            "b": {"type": "number", "description": "Second number"},
        },
        "required": ["a", "b"],
        "additionalProperties": False,
    },
    "strict": True,
}
div_tool = {
    "type": "function",
    "name": "divide",
    "description": "Divide two numbers and return the answer.",
    "parameters": {
        "type": "object",
        "properties": {
            "a": {"type": "number", "description": "Numerator (dividend)"},
            "b": {"type": "number", "description": "Denominator (divisor)"},
        },
        "required": ["a", "b"],
        "additionalProperties": False,
    },
    "strict": True,
}
def add(a, b):
    return a + b

```


Output

```
????????????????????????????????????????????????????????????
START: User question
What is 1247 * 83 + 19 / 3.7?
????????????????????????????????????????????????????????????
????????????????????????????????????????????????????????????
ROUND 1: Model response items
????????????????????????????????????????????????????????????
[0] type = reasoning
[1] type = function_call
name      = multiply
call_id   = call_s9u1bQiyUF7MtUpSrezpuY9K
arguments(raw JSON string) = {"a":1247,"b":83}
[2] type = function_call
name      = divide
call_id   = call_n2NsAEFE5uN5ZESTaF80E4xu
arguments(raw JSON string) = {"a":19,"b":3.7}
? Model requested tool calls:
- multiply({"a":1247,"b":83}) [call_id=call_s9u1bQiyUF7MtUpSrezpuY9K]
- divide({"a":19,"b":3.7}) [call_id=call_n2NsAEFE5uN5ZESTaF80E4xu]
? Executing tools locally (your server/app):
? multiply(**{'a': 1247, 'b': 83}) -> 103501
? divide(**{'a': 19, 'b': 3.7}) -> 5.135135135135135
? Sending tool outputs back to the model:
{'type': 'function_call_output', 'call_id': 'call_s9u1bQiyUF7MtUpSrezpuY9K',
'output': '{"ok": true, "result": 103501}'}
{'type': 'function_call_output', 'call_id': 'call_n2NsAEFE5uN5ZESTaF80E4xu',
'output': '{"ok": true, "result": 5.135135135135135}'}
????????????????????????????????????????????????????????????
ROUND 2: Model response items
????????????????????????????????????????????????????????????
[0] type = reasoning
[1] type = function_call
name      = add
call_id   = call_0Up29xZFdxXKATpZRhvMly8u
arguments(raw JSON string) = {"a":103501,"b":5.135135135135135}
? Model requested tool calls:
- add({"a":103501,"b":5.135135135135135}) [call_id=call_0Up29xZFdxXKATpZRhvMly8u]
? Executing tools locally (your server/app):
```

[Output continues below...]

```
? add(**{'a': 103501, 'b': 5.135135135135135}) -> 103506.13513513513
? Sending tool outputs back to the model:
{'type': 'function_call_output', 'call_id': 'call_0Up29xZFdxXKATpZRhVMly8u',
'output': '{"ok": true, "result": 103506.13513513513}'}
????????????????????????????????????????????????????????
ROUND 3: Model response items
????????????????????????????????????????????????????????
[0] type = message
content = [ResponseOutputText(annotations=[], text='The result is 103,506.13513513513.',
type='output_text', logprobs=[])]
? No function calls. Final assistant text:
The result is 103,506.13513513513.
```